

Part 2 - Python Programming: Methodology and Findings Report

Smart City Traffic Intelligence Capstone

1. Methodology

Part 2 converted the exploratory work from Part 1 into a reproducible Python workflow. The solution is organised into a data pipeline, feature engineering module, visualisation module, and command-line mini application. Logging is used throughout so that each processing step can be traced and reviewed.

Data pipeline

- The raw CSV is loaded with exception handling and validated against the expected nine-column schema before further processing.
- Categorical fields are standardised and date_time is parsed and validated.
- Seventeen exact duplicate rows are removed, reducing the dataset from 48,204 to 48,187 rows.
- Ten invalid 0 K temperature values are imputed using valid monthly medians: 266.83 K for January and 266.51 K for February.
- One extreme rainfall value is replaced with the July median of positive rainfall observations, 1.09 mm.
- The cleaned output is validated and saved as data/processed/traffic_clean_part2.csv.

Feature engineering

- Time features include hour, day_of_week, and weekend.
- Cyclical hour features hour_sin and hour_cos preserve the circular relationship between 23:00 and 00:00.
- Weather indicators is_clear and has_rain are added, while weather_main is one-hot encoded into 11 binary columns.
- temp and traffic_volume are standardised into scaled numerical features.
- A data-driven congestion_category is created from traffic-volume quartiles.
- The engineered dataset contains 48,187 rows and 30 columns and is saved as data/processed/traffic_features_part2.csv.

Logging and reproducibility

Each module uses logging.getLogger(__name__). The log formatter records timestamp, level, module, function, and message. INFO records normal milestones, WARNING records recoverable data-quality changes, ERROR records failures or invalid CLI input, and DEBUG is reserved for intermediate values such as scaling statistics and congestion thresholds. Logs are written to part2_python/pipeline.log.

2. Key Findings

Area	Finding
Hourly traffic	Traffic is lowest overnight, rises sharply from about 05:00-07:00, and reaches its highest average around 16:00.
Weekday vs weekend	Average weekday traffic is about 3,533 vehicles, compared with about 2,571 on weekends.
Weather	Cloudy conditions have the highest average traffic at about 3,618 vehicles, while Squall conditions have the lowest at about 2,062.
CLI high-traffic query	The five highest average traffic hours are 16:00, 17:00, 15:00, 14:00 and 07:00.
Specific timestamp query	For 2016-07-11 17:00, the application returns traffic volume 5,535, weather Rain, and temperature 302.11 K.

Power BI versus Python note

Part 1 Task 4.2C was calculated in Power BI using the raw 48,204-row dataset, which still included 17 exact duplicate records. Part 2 Task 3.3 was calculated in Python using the cleaned 48,187-row dataset after those duplicates were removed. This explains the small decimal-level differences in the reported average traffic values.

3. Mini Application

The mini traffic analytics application supports three commands: compare-days, high-traffic, and traffic-at "YYYY-MM-DD HH:MM". Invalid commands are caught by a custom argparse error handler and logged as ERROR messages. Invalid date/time input is also logged clearly without an unhandled traceback. print() is used only for direct user-facing CLI results.

4. Implications

- Time of day is a stronger operational signal than weather alone for managing this corridor.
- Weekday demand is materially higher than weekend demand, supporting time- and day-specific traffic-management strategies.
- The 16:00 peak and other high-demand hours can be used for traveller messaging, signal timing review, or capacity planning.
- Weather should be treated as a supporting factor rather than a standalone predictor of congestion.
- The reproducible pipeline and logged processing history provide a reliable foundation for Part 3 machine-learning work.

5. Conclusion

Part 2 successfully transformed the raw traffic dataset into a cleaned, feature-engineered, visualised, and queryable Python workflow. The pipeline runs end-to-end, produces auditable logs, saves reusable processed datasets and figures, and provides a simple CLI for traffic queries. The resulting outputs are suitable for direct reuse in Part 3 supervised, unsupervised, deep-learning, explainability, recommendation, and MLOps tasks.